École 2600 — Module Noyau Linux

Rootkit rk2600

Rapport de conception et mise en œuvre

Groupe de 4 étudiants — 2026

Thomas Abreu Dias · Sofiane Belkessa · Vadim Ablancourt · Cornelius Bircu

1. Introduction

Ce rapport décrit la conception et la mise en œuvre du rootkit rk2600, développé dans le cadre du projet offensif de l'école 2600. L'idée centrale était de construire un module noyau Linux qui permette à un programme compagnon d'obtenir des privilèges élevés, tout en rendant son existence aussi difficile que possible à détecter.

Le contexte posé par le sujet : on dispose d'un accès initial sur la machine cible, et le noyau Linux en place accepte le chargement de modules non signés. À partir de là, le rootkit doit pouvoir s'installer discrètement, fournir ses fonctionnalités, et se recharger automatiquement à chaque démarrage.

Quelques contraintes ont guidé les choix d'implémentation : on ne détourne pas le syscall kill, tous les identifiants du code source sont en anglais, et le code respecte les recommandations de l'ANSSI en matière de codage C sécurisé. Le code est entièrement original, aucun copié-collé depuis d'autres projets existants.

2. Architecture générale

Le projet s'organise autour de trois composants. Le module noyau (rk2600.ko) est la pièce centrale, il tourne en ring 0 et fournit toutes les capacités offensives. Le programme compagnon (companion) est l'outil que l'attaquant utilise depuis l'espace utilisateur. L'infrastructure de test repose sur QEMU avec une image Alpine Linux construite par des scripts dédiés.

Le module et le compagnon communiquent via deux canaux distincts, ce qui offre de la flexibilité selon la situation.

2.1 Canaux de communication

Le canal principal est un misc device exposé sous /dev/.rk_ctl avec une interface ioctl. Le préfixe point le rend invisible à un ls ordinaire. L'alternative est une entrée /proc/.rk en écriture seule, qui accepte des commandes texte simples, plus pratique pour interagir directement depuis un shell sans compiler quoi que ce soit.

On a retenu le misc device plutôt qu'un char device classique parce que ça évite de gérer un numéro majeur alloué statiquement. C'est plus propre et moins susceptible d'entrer en conflit avec d'autres drivers présents sur le système.

2.2 Composants du projet

Composant	Localisation	Rôle
Module rk2600	/lib/modules/ <version>/extra/	Noyau, fonctionnalités offensives
Companion CLI	espace utilisateur	Interface de commande
Misc device	/dev/.rk_ctl	Canal ioctl principal
Entrée proc	/proc/.rk	Canal texte alternatif
Script persistence	/etc/local.d/snd-compat.start	Chargement automatique au boot

3. Module noyau

3.1 Communication via ioctl

Le protocole ioctl est défini dans le header partagé rk2600_protocol.h, ce qui permet au module et au compagnon d'utiliser exactement les mêmes définitions sans aucune duplication. Six commandes sont disponibles :

Commande ioctl	Direction	Effet
RK2600_IOC_ELEVATE	_IO	Élévation de privilèges
RK2600_IOC_STATUS	_IOR	Retourne l'état du module
RK2600_IOC_HIDE	_IOW	Ajoute un nom à masquer
RK2600_IOC_UNHIDE	_IOW	Retire un nom de la liste
RK2600_IOC_MODHIDE	_IO	Cache le module lui-même
RK2600_IOC_MODSHOW	_IO	Révèle le module

3.2 Interface /proc

Une entrée /proc/.rk est créée en mode 0200 (écriture seule). Elle accepte les commandes suivantes :

- +nom : ajoute nom à la liste des entrées masquées
- -nom : retire nom de la liste
- hide / unhide : masque ou révèle le module lui-même
- flush : vide entièrement la liste des noms cachés

Cette entrée est elle-même filtrée par le hook proc_filldir décrit dans la section suivante, ce qui la rend invisible depuis ls /proc.

3.3 Masquage des fichiers dans le système de fichiers

Le principe est de remplacer le pointeur iterate_shared dans la structure file_operations du répertoire racine /. Ce hook intercepte les appels de

listing de répertoire au niveau VFS, concrètement, tout ce qui passe par getdents64 finit par passer ici.

La difficulté est que ces structures file_operations se trouvent en mémoire en lecture seule. Pour les modifier, on désactive temporairement la protection en écriture du noyau en effaçant le bit WP du registre CR0 :

```
asm volatile("mov %%cr0, %0" : "=r"(cr0));
asm volatile("mov %0, %%cr0" : : "r"(cr0 & ~X86_CR0_WP) : "memory");
root_fops->iterate_shared = rk_root_iterate;   /* injection du hook */
asm volatile("mov %0, %%cr0" : : "r"(cr0) : "memory");
```

La liste des noms à masquer est gérée par une liste chaînée kernel protégée par un spinlock. À chaque appel filldir, si le nom correspond à une entrée de la liste, elle est simplement filtrée, l'appelant ne la voit jamais. Résultat : ls, find et tout outil basé sur getdents64 ne voient pas les fichiers cachés.

Un hook identique est appliqué sur /proc pour masquer l'entrée .rk de la liste des fichiers du pseudo-système procfs, et sur /dev pour masquer le misc device .rk_ctl. Le rootkit reste invisible à un ls /proc, ls /dev et à tout outil basé sur getdents64 s'appliquant à ces répertoires. Le masquage d'un PID complet (invisibilité d'un processus à ps) n'est pas implémenté dans la version actuelle, voir § 9 Limites connues.

3.4 Masquage du module lui-même

Pour que le module ne soit plus visible dans lsmod, on le retire de la liste chaînée des modules noyau via list_del_init() sur THIS_MODULE->list. Un pointeur vers le nœud précédent (saved_prev) est conservé pour permettre la réinsertion à la demande via RK2600_IOC_MODSHOW. L'entrée sysfs /sys/module/rk2600/ n'est pas retirée dans la version actuelle, voir § 9 Limites connues.

Cette opération est déclenchée automatiquement dans rk_init(), dès le chargement du module, il disparaît des outils de diagnostic habituels.

3.5 Interception des syscalls read, write et sendfile64

Au-delà du masquage par listing de répertoire, le rootkit doit également rester discret au niveau du contenu du script de persistance : un administrateur qui ferait cat /etc/local.d/snd-compat.start pour inspecter les scripts du runlevel local verrait immédiatement la ligne insmod trahissant le chargement du module. Pour y remédier, trois syscalls sont détournés : read (lecture filtrée), write (écriture ignorée, protège le script contre la réécriture) et sendfile64 (refus avec -EINVAL, bloque la copie zero-copy du script). Les trois hooks sont installés via le mécanisme ftrace décrit ci-dessous.

Récupération de la table des syscalls.

Le symbole sys_call_table n'est plus exporté depuis Linux 2.6. Il reste cependant accessible via kallsyms_lookup_name(), elle-même non exportée depuis la version 5.7. Le module utilise le mécanisme de kprobes comme oracle de résolution : une probe est enregistrée avec .symbol_name = "kallsyms_lookup_name", ce qui permet de lire l'adresse résolue dans kp.addr, puis la probe est désenregistrée. Le pointeur obtenu est ensuite appelé pour résoudre sys_call_table.

```
struct kprobe kp = { .symbol_name = "kallsyms_lookup_name" };
register_kprobe(&kp);
klnfn = (kallsyms_lookup_name_fn)kp.addr;
unregister_kprobe(&kp);
rk_sys_call_table = (unsigned long *)klnfn("sys_call_table");
```

Patch de l'entrée __NR_read.

L'entrée correspondant à read est remplacée par rk_sys_read, après désactivation temporaire du bit CR0.WP (même mécanisme qu'au § 3.3). Le pointeur original est sauvegardé dans orig_sys_read pour pouvoir être restauré au rmmod.

Logique de filtrage.

L'identification du script de persistance ne repose pas sur une comparaison de chemin textuel (d_path() serait coûteux à chaque read et vulnérable aux liens symboliques et au bind-mount). Au premier appel, kern_path() est utilisé une fois pour résoudre /etc/local.d/snd-compat.start en couple (inode, dev) stocké dans persist_ino / persist_dev. Chaque appel ultérieur à fd_is_persist() compare simplement f->f_inode->i_ino et f->f_inode->i_sb->s_dev à ces valeurs mémorisées, sans reconstruction de chemin. rk_sys_read appelle d'abord le read original, puis si le fd correspond au script, le buffer utilisateur est recopié, les lignes contenant insmod ou le nom rk2600 sont retirées, et le buffer réduit est réécrit à l'utilisateur. Le nombre d'octets rendu reflète la taille filtrée. rk_sys_write, lui, retourne count sans jamais appeler l'original quand le fd correspond : l'écriture est silencieusement simulée, le script reste intact sur disque. rk_sys_sendfile64 refuse avec -EINVAL dès que l'in_fd pointe sur le script.

```
# avant le hook :
#!/bin/sh
# Sound compatibility layer
printk_lvl=$(cat /proc/sys/kernel/printk | cut -f1)
echo 1 > /proc/sys/kernel/printk
/sbin/insmod /lib/modules/.../rk2600.ko 2>/dev/null
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1
```

```
# après le hook, du point de vue utilisateur :
#!/bin/sh
# Sound compatibility layer
printk_lvl=$(cat /proc/sys/kernel/printk | cut -f1)
echo 1 > /proc/sys/kernel/printk
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1
```

Choix du syscall.

Le sujet interdit explicitement le détournement de kill. read est retenu pour deux raisons : il est appelé par pratiquement tous les outils d'inspection (cat, less, head, grep, les éditeurs), et le sujet le mentionne lui-même comme exemple d'approche crédible (p. 5).

Limites.

Les hooks couvrent read, write et sendfile64, mais pas les variantes pread64, readv, preadv, preadv2, splice ni l'accès mmap. Une démonstration vim /etc/local.d/snd-compat.start peut donc révéler la ligne cachée si l'éditeur utilise mmap. Pour le périmètre du projet, ce compromis est acceptable.

4. Élévation de privilèges

La commande ELEVATE repose sur prepare_kernel_cred(NULL), qui génère des credentials kernel avec uid=0, gid=0 et toutes les capabilities POSIX. Ces credentials sont appliqués au processus appelant via commit_creds(). Simple, efficace, et stable d'une version de noyau à l'autre.

```
new = prepare_kernel_cred(NULL);
commit_creds(new);
```

Du côté du compagnon, après l'ioctl, on vérifie que geteuid() == 0 avant de lancer le shell /bin/sh. Le device /dev/.rk_ctl est créé avec les permissions 0666 précisément pour permettre cet appel depuis un compte non privilégié, c'est l'objectif premier.

Cette permission large signifie que n'importe quel utilisateur local peut invoquer RK2600_IOC_ELEVATE et obtenir un shell root. Dans le cadre du projet, c'est le comportement attendu : la démonstration doit pouvoir être rejouée depuis le compte user sans manipulation préalable. Dans un déploiement réel, ce choix serait remplacé par une authentification par défi/réponse, voir § 9 Limites connues.

On a préféré cette approche à une manipulation directe des champs uid dans task_struct parce que prepare_kernel_cred() est une API interne

documentée, plus résistante aux changements structurels entre versions de noyau.

5. Persistance au redémarrage

La persistance passe par le mécanisme local.d d'OpenRC, natif à Alpine Linux. La commande companion install effectue les étapes suivantes :

- Copie rk2600.ko dans /lib/modules/<version>/extra/
- Crée /etc/local.d/snd-compat.start, nom volontairement camouflé en compatibilité audio
- Active le service local : rc-update add local default
- Charge immédiatement le module via insmod

Le script de persistence réduit le niveau de log du noyau à 1 avant l'insmod, puis le restaure. Le ring buffer dmesg est vidé. Résultat : aucun message ne s'affiche sur la console au boot.

```
echo 1 > /proc/sys/kernel/printk
/sbin/insmod /lib/modules/.../rk2600.ko 2>/dev/null
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1
```

Cinq noms sont ajoutés à la liste de masquage par rk_init() dès le premier chargement : snd-compat.start (nom du script), rk2600.ko (fichier du module), rk2600 (nom du module affiché par les outils sysfs), .rk (entrée procfs de contrôle) et .rk_ctl (misc device sous /dev). Un utilisateur faisant ls /etc/local.d/, ls /lib/modules/.../extra/, ls /proc ou ls /dev ne les verra pas.

6. Programme compagnon

Le compagnon est une CLI en C qui communique avec le module via /dev/.rk_ctl. On a choisi de rester sur une implémentation simple et sans dépendances externes, uniquement la libc standard. Cela facilite la portabilité et la compilation croisée.

Commande	Arguments	Description
status	—	Affiche la version API et l'état de masquage du module
privesc	—	Élève les privilèges et ouvre un shell root
hide	<nom>	Masque l'entrée FS <nom> dans tous les listings
unhide	<nom>	Révèle l'entrée FS <nom>

install	[ko]	Installe le module et configure la persistance
uninstall	—	Désinstalle le module et supprime les fichiers de persistance
test	—	Lance la suite de tests automatisés (12 tests)

La validation des arguments est stricte : longueur max de 255 caractères (RK2600_NAME_MAX), refus des chemins contenant /, refus de . et ... La copie du .ko lors de l'installation est faite par le compagnon lui-même via une fonction copy_file() interne, sans faire appel à cp.

7. Environnement de test

L'environnement repose sur QEMU avec une image disque Alpine Linux. Tout est scripté, reconstruire l'environnement from scratch se fait en une seule commande via le Makefile racine.

Script	Rôle
build_kernel.sh	Compilation croisée du noyau Linux (cible x86_64)
build_rootfs.sh	Construction du rootfs BusyBox minimal
build_disk.sh	Création de l'image disque bootable Alpine + GRUB BIOS
run.sh	Lancement de la VM QEMU avec les paramètres adaptés

L'image est générée en format raw par build_disk.sh, puis convertie en QCOW2 compressé pour le livrable via la cible make qcow2. La conversion n'est pas chaînée aux cibles make disk ou make all pour éviter une conversion coûteuse à chaque itération de build : elle est lancée explicitement, une fois, au moment du dépôt.

```
make qcow2   # exécute : qemu-img convert -c -O qcow2 infra/disk.img rk2600.qcow2
```

La taille du rendu passe typiquement de 1 Go (raw) à ~30 Mo (QCOW2 compressé).

La LFS contient deux comptes :

- root — pour charger le module (insmod rk2600.ko)
- user — pour démontrer l'usage du rootkit via le programme compagnon

Les mots de passe sont documentés dans la documentation utilisateur (docs/USER.md) fournie avec le rendu.

8. Scénarios d'accès initial sur systèmes non signés

Le sujet pose comme hypothèse l'existence d'un accès initial sur une machine dont le noyau accepte les modules non signés. Cette section bonus justifie que cette hypothèse décrit des situations réelles et pas un cas d'école. Cinq scénarios sont présentés ci-dessous, du plus au moins représenté dans le parc Linux déployé.

8.1 IoT grand public et périphériques réseau

Routeurs sous OpenWrt, NAS Synology/QNAP, box opérateur, caméras IP, TV connectées, décodeurs : la plupart tournent sous un noyau Linux custom compilé par le constructeur, souvent avec CONFIG_MODULE_SIG=n pour éviter la gestion d'une PKI interne et accélérer les cycles de mise à jour. La signature se fait en général au niveau du firmware entier plutôt qu'au niveau des modules individuels.

Vecteur d'accès initial typique : exploitation d'une CVE dans l'interface web d'administration ou d'un service UPnP exposé (classe Mirai). Une fois le shell root obtenu, le dépôt d'un .ko custom est trivial.

8.2 Systèmes industriels et médicaux (OT)

PLC sous Linux, superviseurs SCADA, équipements d'imagerie médicale, imprimantes multifonction haut-de-gamme : parc gelé sur des branches noyau anciennes (3.x, 4.x) pour des raisons de certification ou de stabilité. Secure Boot rarement activé car non requis par la norme applicable et jugé incompatible avec les procédures de maintenance sur site.

Vecteur d'accès initial typique : intervention physique d'un technicien de maintenance avec clé USB, ou prise de contrôle via un tunnel VPN d'assistance distante mal segmenté.

8.3 Workstation de développeur et runner CI auto-hébergé

Postes où Secure Boot est désactivé par confort (dual-boot Windows, drivers nvidia non signés, kernels compilés par l'utilisateur). Les runners CI d'organisations qui refusent les GitHub-hosted runners pour des raisons de confidentialité : souvent des VM Linux allégées, sans Secure Boot, où les pipelines s'exécutent en root ou via sudo sans mot de passe.

Vecteur d'accès initial typique : compromission supply-chain d'une dépendance (npm install, pip install, cargo install) qui exécute un script post-install exfiltrant un token CI, ou exécution d'un job fetché depuis un fork malveillant.

8.4 Secure Boot actif mais MOK compromise

Systèmes où l'admin a enrôlé sa propre clé MOK (Machine Owner Key) pour charger un driver tiers (nvidia-dkms, zfs-dkms, vmware-modules,

VirtualBox). La clé privée se retrouve souvent stockée en clair sur le disque dans /var/lib/dkms/mok.key ou dans /root/.mok/. Son vol permet de signer un .ko malveillant qui passera la vérification mokutil.

Vecteur d'accès initial typique : élévation locale via une CVE classique (polkit, pkexec, kernel), puis lecture directe de la clé MOK. Ce scénario rejoint le profil 8.3 mais avec signature propre.

8.5 Datacenter et distributions internes

Hébergeurs et grands opérateurs cloud qui maintiennent leur propre branche noyau interne. Secure Boot désactivé sur les hyperviseurs et sur une partie du parc guest. Les kernels internes n'ont pas toujours de signature module activée pour éviter la double gestion de clés (constructeur + exploitant).

Vecteur d'accès initial typique : compromission d'un compte administrateur par phishing, puis latéralisation vers les machines de build/déploiement.

8.6 Synthèse

Le modèle d'attaque posé par le sujet, module non signé chargé après un accès initial, couvre une part significative du parc Linux déployé, en particulier hors serveurs grand public sous distribution commerciale avec Secure Boot. Les défenses complémentaires qui réduisent l'impact du rootkit sont : IMA/EVM (intégrité du filesystem), kernel lockdown (interdit les écritures en /dev/mem, kprobes, module load sans signature même sans Secure Boot), auditd avec règles sur init_module / finit_module, et kernel runtime integrity monitoring (KASLR, KPTI, CET).

9. Limites connues

Le rootkit est fonctionnel sur la configuration de référence (Alpine Linux + noyau Linux 6.1.188 x86_64), mais présente plusieurs limites dont certaines sont volontaires.

9.1 Posture de sécurité du device de contrôle

/dev/.rk_ctl est en mode 0666. Tout utilisateur local peut donc appeler RK2600_IOC_ELEVATE et obtenir un shell root. C'est un choix volontaire dicté par le besoin de rejouer la démonstration depuis le compte user. Une version renforcée imposerait un défi/réponse : le compagnon lirait un nonce via RK2600_IOC_STATUS, calculerait un HMAC avec une clé partagée, et le module refuserait ELEVATE tant que le HMAC n'est pas fourni en retour.

9.2 Portée du hook VFS

Le hook iterate_shared n'est installé que sur les file_operations de /, /proc et /dev. Un listing de /mnt/usb/ ou de n'importe quel autre point de montage

non associé à ces trois répertoires n'est pas filtré. Une évolution naturelle serait d'installer le hook directement sur __NR_getdents64 (comme pour read), ce qui couvrirait tous les systèmes de fichiers.

9.3 Variantes de lecture non couvertes

Les variantes pread64, readv, preadv2 et l'accès mmap au script de persistance ne sont pas filtrés. Un éditeur qui mmap le fichier (vim en mode par défaut) verrait les lignes trahissantes.

9.4 Écritures hors write/sendfile64

Les hooks write et sendfile64 protègent le script contre un cat > /etc/local.d/snd-compat.start ou une copie zero-copy, mais pas contre les autres voies d'écriture : pwrite64, writev, pwritev, splice en sortie, ou un éditeur qui fait un unlink + rename atomique (vim avec backup) contournent le filtre et invalideraient la persistance au prochain boot. Un hook sur __NR_unlinkat et __NR_renameat2 serait nécessaire pour couvrir ce dernier cas.

9.5 Fuite via /proc/kallsyms

Les symboles du module (rk_sys_read, rk_root_iterate, rk_ioctl, rk_fops, etc.) restent lisibles dans /proc/kallsyms et /proc/kcore. Une analyse mémoire ou un grep rk /proc/kallsyms révèle la présence du rootkit. Un filtrage dédié de ces deux fichiers, au niveau du hook read, permettrait de refermer cette brèche.

9.6 Mécanisme CR0.WP

Le patch de pages read-only via mov %cr0 fonctionne sur le noyau 6.1.188 utilisé pour la démonstration. Les noyaux plus récents durcis par CONFIG_X86_CPA_STATICCHECKS ou par CFI Clang peuvent refuser ou détecter cette manipulation. Un portage ftrace-based du hooking syscall (via -pg/-mfentry) serait la solution d'évolution propre et est référencé dans le cours 3/4.

9.7 Architecture

Le code assume x86_64 : registre pt_regs->di pour le premier argument de syscall, instruction mov %cr0, numéro __NR_read x86. Le portage ARM64 nécessiterait une couche d'abstraction sur pt_regs et un mécanisme de bypass d'écriture spécifique à l'ABI (pas de CR0).

9.8 Détection via l'intégrité des file_operations

Un outil de détection côté noyau qui aurait un instantané des pointeurs f_op->iterate_shared au boot comparerait et détecterait la substitution. En l'état, rien dans le rootkit ne masque cette modification. Une évolution via ftrace rendrait la détection plus difficile car f_op ne serait pas modifié.

10. Build et compatibilité

10.1 Noyau cible

Le module est compilé contre Linux 6.1.188 (branche longterm, dernière révision stable disponible au début du projet). Le .ko doit être compilé contre exactement la même version et le même .config que le noyau installé dans la LFS. Toute divergence produit un Invalid module format à l'insmod.

Le .config fourni désactive explicitement :

- CONFIG_MODULE_SIG (chargement de modules non signés, cœur du sujet)
- CONFIG_LOCKDOWN_LSM (accès libre à kprobes et à l'écriture mémoire noyau)
- CONFIG_RANDOMIZE_BASE (profil debug uniquement, pour les breakpoints GDB)

10.2 Compilation

Depuis la racine du dépôt :

```
make kernel PROFILE=base       # compile le noyau
make module                 # compile rk2600.ko contre infra/build
make companion                # compile le binaire user
make disk                  # construit la LFS Alpine + GRUB BIOS
make qcow2                  # convertit en rk2600.qcow2 compressé (livrable)
```

10.3 Chargement et symboles

Le module nécessite MODULE_LICENSE("GPL") pour obtenir l'accès au symbole non-exporté kallsyms_lookup_name via kprobe. Sans licence GPL, le noyau refuse la résolution.

11. Répartition des tâches

Chaque membre a eu un domaine principal de responsabilité, avec des zones de collaboration sur les points d'interface entre composants.

Membre	Domaine principal	Tâches réalisées
Thomas Abreu Dias	Module noyau Masquage FS	Hook iterate_shared sur / Liste chaînée des noms cachés Bypass write-protect (CR0) Hook iterate_shared sur /proc
Sofiane Belkessa	Module noyau ; Comm. & privesc	Misc device & interface ioctl

		Protocole partagé (rk2600_protocol.h) Élévation de privilèges Masquage du module (sysfs/kobj)
Vadim Ablancourt	Programme compagnon, read hook & tests	CLI compagnon (toutes les commandes) Interface /proc/.rk Hook sys_read via kallsyms_lookup_name + kprobe Validation des arguments Tests d'intégration Documentation utilisateur (docs/USER.md)
Cornelius Bircu	Infrastructure & build system	Scripts de build (noyau, rootfs, disque) Image Alpine + GRUB BIOS Cross-compilation QEMU Script de persistence OpenRC

12. Tableau de ratio de contribution

Un ratio de 100% correspond à une contribution normale attendue. La note de groupe est multipliée par ce ratio pour obtenir la note individuelle. Les membres à 0% obtiennent 0.

Membre	Ratio (%)
Thomas Abreu Dias	100 %
Sofiane Belkessa	100 %
Vadim Ablancourt	100 %
Cornelius Bircu	100 %

Note : ces ratios sont à valider et ajuster par le groupe avant le rendu final.